Lab-2

Implementation of Symmetric & Asymmetric Cryptography (AES & RSA)

Tools Used

- OpenSSL
- Linux OS (Kali / Ubuntu)

Theory

Symmetric cryptography uses a **single shared key** for encryption and decryption (e.g., AES).
Asymmetric cryptography uses a **public–private key pair** (e.g., RSA).
AES is fast and efficient for large data, while RSA is used for secure key exchange.

Part A: AES (Symmetric Encryption)

Step 1: Create a Plaintext File

```
echo "Confidential Data for AES Encryption" > plain.txt
```

Step 2: Encrypt Using AES

```
openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in plain.txt -out aes_encrypted.bin
```

Option	Why it's needed
-pbkdf2	Uses modern key derivation instead of deprecated EVP_BytesToKey
-iter 100000	Makes password brute-force attacks harder
-salt	Adds randomness so same password ≠ same ciphertext
-aes-256-cbc	Strong symmetric encryption

Step 3: Decrypt Using AES

```
openssl enc -aes-256-cbc -d -salt -pbkdf2 -iter 100000 -in aes_encrypted.bin -out decrypted.txt
```

CBC = **C**ipher **B**lock **C**haining Mode

Part B: RSA (Asymmetric Encryption)

Step 4: Generate RSA Private Key

```
openssl genrsa -out rsa_private.key 2048
```

Step 5: Generate RSA Public Key

```
openssl rsa -in rsa_private.key -pubout -out rsa_public.key
```

Step 6: Encrypt Using Public Key

```
openssl pkeyutl -encrypt -inkey rsa_public.key -pubin -in plain.txt -out rsa_encrypted.bin
```

Step 7: Decrypt Using Private Key

```
openssl pkeyutl -decrypt -inkey rsa_private.key -in rsa_encrypted.bin -out rsa_decrypted.txt
```

Hash Functions, Digital Signatures & Public Key Infrastructure (PKI)

Tools Used

- OpenSSL
- GPG (GNU Privacy Gauard)

Theory

Hash functions ensure **data integrity**.
Digital signatures provide **authentication, integrity, and non-repudiation**.
PKI uses certificates issued by a **Certificate Authority (CA)** to establish trust.

Part A: Hash Generation (MD5 & SHA)

Step 1: Create File

```
echo "Integrity Check File" > data.txt
```

Step 2: Generate MD5 Hash

```
md5sum data.txt
```

Step 3: Generate SHA-256 Hash

```
sha256sum data.txt
```

Part B: Digital Signature Using GPG

Step 4: Generate GPG Key Pair

```
gpg --full-generate-key
```

Step 5: Digitally Sign the File

```
gpg --sign data.txt
```

Step 6: Verify Digital Signature

```
gpg --verify data.txt.gpg
```

🔏 Digital Signature Using OpenSSL

- Step 1: Generate Private Key

```
openssl genrsa -out private.key 2048
```

- Step 2: Generate Public Key

```
openssl rsa -in private.key -pubout -out public.key
```

- Step 3: Create a File

```
echo "This is a confidential message" > message.txt
```

- Step 4: Create Digital Signature

```
openssl dgst -sha256 -sign private.key -out signature.bin message.txt
```

✓ signature.bin is the **digital signature**

- Step 5: Verify Digital Signature

```
openssl dgst -sha256 -verify public.key -signature signature.bin message.txt
```

Part C: PKI – Digital Certificate Creation

Step 7: Generate Private Key

```
openssl genrsa -out ca_private.key 2048
```

Step 8: Create Self-Signed Certificate

```
openssl req -x509 -new -nodes -key ca_private.key -sha256 -days 365 -out ca_certificate.crt

openssl x509 -in ca_certificate.crt -text -noout

openssl x509 -in ca_certificate.crt -noout -subject -issuer
```

👉 If **subject == issuer**, it means it's a **self-signed CA certificate**.